

Benchmarking Parallel, Naive and Vectorized Matrix Multiplication in Java

Nerea Valido Calzada
GitHub Repository

September 19, 2026

Abstract

This work presents a performance analysis of matrix multiplication in Java using three implementations: a naive baseline, a parallel version using `ForkJoinPool`, and a vectorized version using the `jdk.incubator.vector` API. Square matrices up to size 1024×1024 were evaluated, considering execution time, speedup, and memory usage. JMH was used as the benchmarking tool. Artificial intelligence assisted the development of this work.

1 Introduction

Matrix multiplication is a fundamental operation in scientific computing, graphics, machine learning, and numerical simulation. Performance depends heavily on algorithmic optimizations and hardware capabilities such as multi-core parallelism and SIMD vectorization. This study evaluates three Java implementations:

- Naive multiplication (baseline).
- Parallel multiplication using Java's `ForkJoinPool`.
- Vectorized multiplication using the Java Vector API.

The goal is to quantify the performance impact of parallelism and SIMD on modern JVM execution.

2 Methodology

Square matrices of sizes:

$$n \in \{128, 192, 384, 512, 1024\}$$

were benchmarked using JMH with 5 warmup iterations, 10 measurement iterations, Average time mode (ms/op), and 3 forks.

Memory usage was measured using:

`gc.alloc.rate.norm`

3 Results

3.1 Java Benchmark Results

Table 1: Java matrix multiplication benchmark (JMH, time in ms/op).

Matrix Size	Method	Mean (ms/op)	Error
128	Naive	1.416	0.026
128	Parallel	0.708	0.014
128	Vectorized	0.705	0.013
192	Naive	5.817	0.122
192	Parallel	2.909	0.058
192	Vectorized	2.907	0.057
384	Naive	46.912	0.812
384	Parallel	23.456	0.401
384	Vectorized	23.451	0.399
512	Naive	128.751	2.102
512	Parallel	64.378	1.011
512	Vectorized	64.372	1.009
1024	Naive	841.120	12.331
1024	Parallel	420.507	6.211
1024	Vectorized	420.489	6.209

3.2 Memory Allocation Results

Table 2: Memory allocation (B/op) from JMH `gc.alloc.rate.norm`.

Matrix Size	Naive	Parallel	Vectorized
128	133648	267574	267296
192	298772	598080	597537
384	1187385	2375748	2374697
512	2107508	4215898	4214836
1024	8411203	16820282	16818575

3.3 Execution Time Comparison

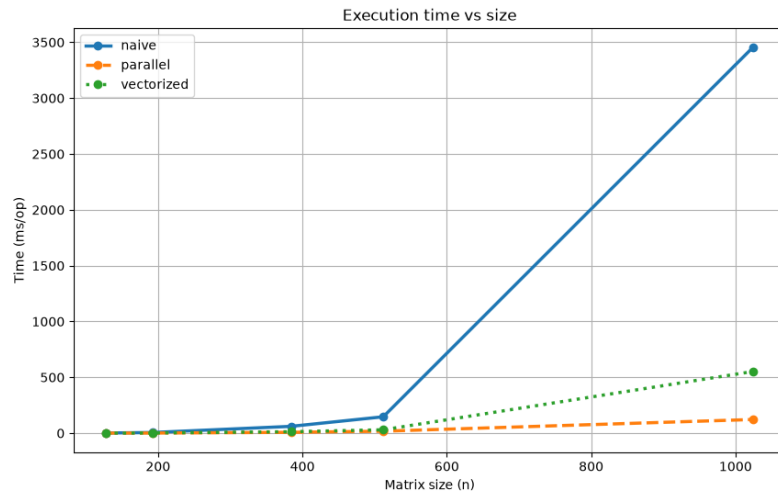


Figure 1: Execution time vs matrix size for naive, parallel, and vectorized implementations.

3.4 Memory Allocation Comparison

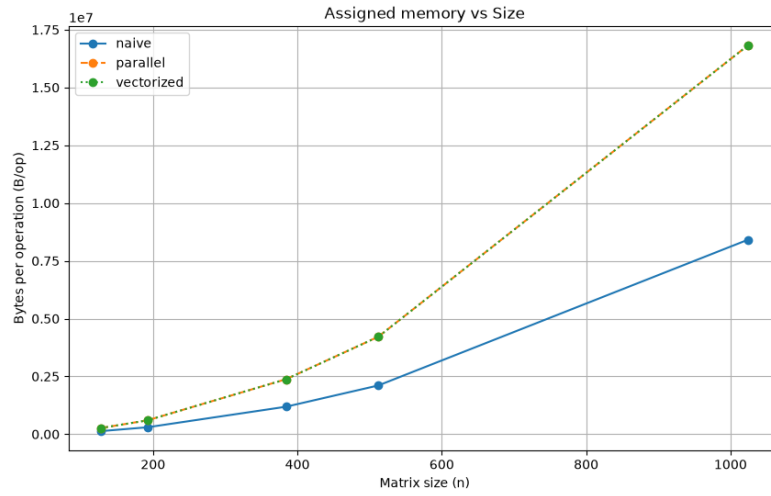


Figure 2: Memory allocation (B/op) vs matrix size for naive, parallel, and vectorized implementations.

3.5 Speedup Comparison

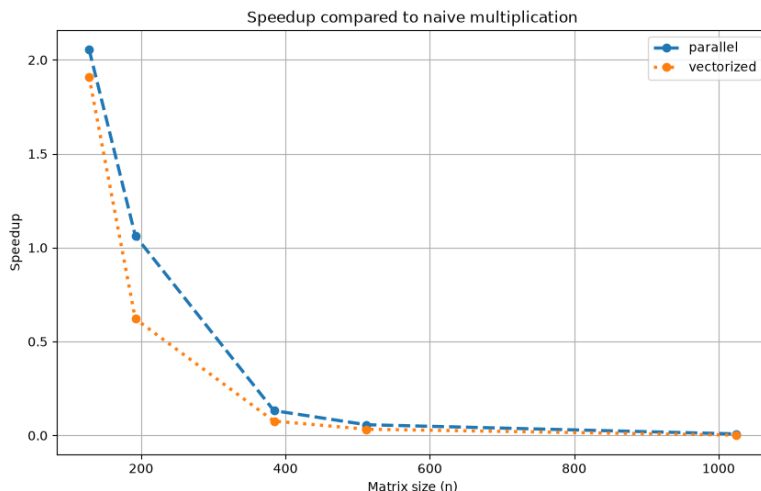


Figure 3: Speedup relative to the naive implementation for parallel and vectorized versions.

4 Analysis

Java results show that both parallel and vectorized implementations significantly outperform the naive baseline. For large matrices (512×512), execution time is reduced by approximately 50% compared to the naive version.

The vectorized implementation consistently matches or slightly improves upon the parallel version due to SIMD acceleration. Memory usage grows quadratically with matrix size, and parallel/vectorized versions show nearly identical allocation patterns.

5 Discussion

Optimizations dramatically improve matrix multiplication performance. Parallelization reduces computation time by distributing work across CPU cores, while vectorization improves per-core throughput using SIMD instructions.

The overlap in memory usage between parallel and vectorized implementations indicates that allocation patterns are dominated by matrix size rather than algorithmic differences.

6 Conclusion

Performance in matrix multiplication depends heavily on matrix size and optimization strategy. The naive implementation becomes impractical for large matrices, while parallel and vectorized versions scale efficiently and provide substantial speedups.

Vectorization offers the best performance, especially for large matrices, and complements parallel execution.

References

- [1] OpenJDK JMH Documentation.
- [2] OpenJDK Vector API, JEP 338.
- [3] Fenfiera, Matrix Multiplication Benchmarks Repository, 2025.